Benchmark Harness: Technical Documentation

Nov 10, 2026 · @Mohamed khalil Ankri

Overview

This repository is a benchmark harness that runs the same four TPC-H analytical queries on PostgreSQL 18.6 and ClickHouse 26.8.36.3, under identical resource limits, at several data sizes, and records how fast each engine answers. It implements the protocol of our M1 report (Sections 5.1–5.6). Scale factor 1 is complete: data loaded, results validated, and timings measured.

The research question is how dataset size affects the relative query performance of the two engines. The harness therefore measures the speedup S = PostgreSQL median time ÷ ClickHouse median time, for each query and each size, and how S changes from one size to the next.

One experiment cycle, repeated for each scale factor (SF 1, 2, 4, 10):

1. **Generate** the TPC-H data with DBGen (`data/sf<N>/*.tbl`).
2. **Load** the same files into both engines, under two physical designs: B (baseline) and D (sorted on ship date).
3. **Validate** that both engines hold identical data and return identical query results.
4. **Record sizes** of every table in both engines.
5. **Measure** cold and warm runs of the four queries, one engine at a time (`scripts/run.py`).
6. **Analyse** medians, speedups, confidence intervals and the hypothesis decision table (`scripts/analyze.py`).
7. **Commit** the raw results and settings to Git.

Three principles hold throughout:

- **Nothing is typed by hand.** Every number comes from a script and lands in a file under `results/`.
- **Same conditions for both engines.** Same CPUs, same memory limit, same data files, same query semantics, one engine running at a time.
- **Validate before timing.** A speed comparison is only meaningful if both engines compute the same answer.

Repository layout

The project lives inside Ubuntu at `~/DB-Benchmark` (from Windows: `\\wsl.localhost\Ubuntu\home\khalil\DB-Benchmark`). Everything except `data/` and `.venv/` is versioned in Git.

Path	Purpose
`docker-compose.yml`	Defines the two engine containers with identical CPU and memory limits, pinned image digests and PostgreSQL settings
`config/clickhouse/bench.xml`	ClickHouse settings (threads, memory, caches disabled), mounted into the container
`requirements.txt`	Python packages for the scripts
`data/sf<N>/`	Generated TPC-H files (`lineitem.tbl`, `orders.tbl`, `part.tbl`); not in Git (several GB)
`sql/postgres/schema.sql`	Creates schema `b` and the three tables (design B)
`sql/postgres/post_load.sql`	Adds primary keys and runs `VACUUM ANALYZE` after loading
`sql/postgres/design_D.sql`	Creates schema `d` with LINEITEM sorted by ship date and a BRIN index
`sql/postgres/q1.sql` … `q14.sql`	The four queries, PostgreSQL dialect
`sql/clickhouse/schema.sql`	Creates database `b` and the three MergeTree tables (design B)
`sql/clickhouse/design_D.sql`	Creates database `d` with LINEITEM sorted by ship date
`sql/clickhouse/q1.sql` … `q14.sql`	The four queries, ClickHouse dialect
`sql/answers/`	TPC-H reference answers for SF 1 (`q1.out`, `q6.out`, `q12.out`, `q14.out`)
`scripts/load_test.sh`	Loads one scale factor into both engines, both designs, and prints data fingerprints
`scripts/check_queries.sh`	Runs the four queries on both engines next to the reference answers

Path	Purpose
`scripts/sizes.sh`	Records table and column sizes, and the physical-order check for design D
`scripts/reset_caches.sh`	Stops an engine, clears the Linux page cache, restarts it
`scripts/run.py`	The measurement runner (cold, warm-up, warm and instrumented runs)
`scripts/analyze.py`	Medians, speedups, bootstrap confidence intervals, decision table, plot
`env/`	Host description, image digests, DBGen version, engine settings dumped at each run
`results/runs.csv`	Raw measurements: one row per timed query execution
`results/ch_query_log.csv`	Rows and bytes ClickHouse actually read, per run
`results/resources.csv`	CPU and memory of the running container, sampled every second
`results/plans/`	Execution plans saved by the instrumented run
`results/summary_<tag>.csv`, `speedups_<tag>.csv`, `decisions_<tag>.csv`, `speedup_<tag>.png`	Analysis outputs per campaign tag
`results/check_sf1.txt`, `load_test_sf1.txt`, `sizes_sf1.txt`	Validation and size evidence for SF 1

The TPC-H generator is cloned separately at `~/tpch-kit`, outside the repository.

Host setup

The experiment runs on a Windows laptop through WSL2, configured so that the two things the protocol depends on work: a container memory limit that includes the page cache, and readable per-container disk counters.

Item	Value
Machine	Intel Core i5-10300H (4 cores, 8 threads), 16 GB RAM, SSD (drive E:)
Linux	Ubuntu 22.04.3 under WSL2, kernel 6.18
Memory for WSL2	12 GB, no swap
cgroup	Version 2, systemd driver
Docker	Docker Engine 29.9.3 installed inside Ubuntu
Ubuntu's virtual disk	Moved to `E:\WSL\Ubuntu\ext4.vhdx` (C: was full)

WSL2 configuration

`C:\Users\<you>\.wslconfig` (Windows side):

```
[wsl2]
memory=12GB
processors=8
swap=0
kernelCommandLine=cgroup_no_v1=all
```

`/etc/wsl.conf` (inside Ubuntu):

```
[boot]
systemd=true

[interop]
appendWindowsPath=false
```

Why each setting matters:

- `memory=12GB` leaves room for an 8 GB container plus Linux, Docker and the scripts; Windows keeps 4 GB.
- `swap=0` prevents Linux from silently using disk as extra memory, which would distort the memory regimes.
- `cgroup_no_v1=all` forces cgroup v2, in which a container's memory limit also covers its page cache and `io.stat` reports its disk reads.
- `systemd=true` lets Docker use the systemd cgroup driver, which places containers at a predictable path: `/sys/fs/cgroup/system.slice/docker-<id>.scope/`.

- `appendWindowsPath=false` stops Ubuntu from picking up the Windows Docker Desktop binary by accident.

Why Docker Engine and not Docker Desktop

Docker Desktop runs containers in a separate hidden Linux distribution, so their cgroup files are not reliably readable from Ubuntu; it also updates itself and has its own resource management. Docker Engine inside Ubuntu avoids all three. Docker Desktop's WSL integration is turned off, and it must stay closed during measurements.

Cache clearing without a password

Cold runs clear the Linux page cache dozens of times. A sudoers rule allows exactly that one command without a password:

```
echo "$USER ALL=(root) NOPASSWD: /usr/bin/tee /proc/sys/vm/drop_caches" |
sudo tee /etc/sudoers.d/dropcaches
sudo chmod 440 /etc/sudoers.d/dropcaches
```

Check that a host is ready

```
stat -fc %T /sys/fs/cgroup                          # cgroup2fs
free -h                                             # about 11.7 GiB, swap 0
docker info | grep -iE "cgroup|operating system" # systemd, 2, Ubuntu
echo 3 | sudo -n tee /proc/sys/vm/drop_caches     # prints 3, no password
prompt
```

The host description is saved in `env/` (`lscpu.txt`, `cpus.txt`, `memory.txt`, `disk.txt`, `kernel.txt`, `docker.txt`, `docker_info.txt`).

Containers and engine configuration

Each engine runs in its own container, pinned to the same four logical CPUs with the same 8 GB memory limit and no swap; only one container runs during any measurement.

Setting	PostgreSQL (`pg`)	ClickHouse (`ch`)
Image	`postgres:18.6@sha256:e3d24f69…3c42`	`clickhouse/clickhouse-server:26.8.36.3@sha256:d426d2f6…aa7a7`
CPUs (`cpuset`)	0-3	0-3
Memory (`mem_limit` = `memswap_limit`)	8 GB	8 GB

Setting	PostgreSQL (`pg`)	ClickHouse (`ch`)
Extra	`shm_size: 2560m`	`nofile` limit 262144
Port	5432	8123 (HTTP), 9000 (native)
Password	`bench`	`bench`
Data volume	`pgdata` → `/var/lib/postgresql`	`chdata` → `/var/lib/clickhouse`
Generated files	`./data` → `/data` (read-only)	`./data` → `/data` (read-only)

The full digests are in `env/images.txt`. Pinning by digest guarantees the exact same software for the whole experiment.

`cpuset 0-3` is two physical cores with their two hyperthreads (`env/cpus.txt`); under WSL2 Windows decides the real mapping, so this is approximate. `shm_size` is needed because PostgreSQL places shared buffers and parallel-query memory in `/dev/shm`; Docker's 64 MB default makes parallel queries fail. That memory counts inside the 8 GB limit.

Engine settings (report Table 5)

Engine	Setting	Value	Reason
PostgreSQL	`max_parallel_workers_per_gather`	3	Leader + 3 workers = 4 CPUs
PostgreSQL	`max_parallel_workers`	4	
PostgreSQL	`shared_buffers`	2GB	25% of the memory budget
PostgreSQL	`effective_cache_size`	6GB	Budget minus shared buffers
PostgreSQL	`work_mem`	256MB	Per hash or sort operation
PostgreSQL	`random_page_cost`	1.1	SSD storage
PostgreSQL	`jit`	on	Default, kept explicitly
ClickHouse	`max_threads`	4	One thread per CPU
ClickHouse	`max_memory_usage`	6 GB	Per-query limit inside the 8 GB budget
ClickHouse	`use_query_cache`	0	No caching of query results

Engine	Setting	Value	Reason
ClickHouse	`use_query_condition_cache`	0	No caching of filter outcomes between runs (enabled by default)
ClickHouse	`use_uncompressed_cache`	0	Default, kept explicitly

PostgreSQL settings are passed as `-c` flags in the compose file; ClickHouse settings are in the `default` profile of `config/clickhouse/bench.xml`. All other settings keep their defaults. `run.py` dumps the full effective settings at the start of every run to `env/settings_<tag>_<engine>_sf<N>.txt`. PostgreSQL 18.6 runs with `io_method = worker` (default).

Start or stop one engine:

```
docker compose up -d postgres      # or clickhouse
docker compose stop postgres
```

Data generation and loading

The same generated files are loaded into both engines, under two physical designs that differ only in physical order and indexing.

Generating the data

DBGen 2.17.3 from the `tpch-kit` repository (commit recorded in `env/dbgen.txt`):

```
cd ~ && git clone https://github.com/gregrahn/tpch-kit.git
cd tpch-kit/dbgen && make MACHINE=LINUX DATABASE=POSTGRESQL
./dbgen -s 1 -f                              # -s = scale factor
mkdir -p ~/DB-Benchmark/data/sf1
mv lineitem.tbl orders.tbl part.tbl ~/DB-Benchmark/data/sf1/
rm -f *.tbl
sed -i 's/|$//' ~/DB-Benchmark/data/sf1/*.tbl  # strip the trailing delimiter
```

Only LINEITEM, ORDERS and PART are kept, because the four queries use no other table. DBGen ends every line with `|`, which both engines would read as an extra empty column, so it is stripped. Expected line counts at SF 1: 6,001,215 / 1,500,000 / 200,000; all scale linearly with SF.

Table definitions

Column type	PostgreSQL	ClickHouse
Keys and integers	`INTEGER`	`Int32`
Money and quantities	`NUMERIC(15,2)`	`Decimal(15,2)`
Dates	`DATE`	`Date`
Text	`VARCHAR(n)`	`String`

Exact decimal types in both engines let results be compared to the cent. ClickHouse uses plain `String`, not `LowCardinality(String)`: that is a ClickHouse-only optimisation, and using it would tune one engine and not the other.

The two physical designs

	Design B (baseline)	Design D (data skipping)
Purpose	Neither engine can skip data on the filtered columns; isolates storage layout and execution	Both engines can skip data on ship-date filters; measures the effect of a sorting key
PostgreSQL	Schema `b`: heap tables in generation order (order key), primary-key B-tree indexes	Schema `d`: `lineitem` copied sorted by `l_shipdate`, primary key, BRIN index on `l_shipdate`; ORDERS and PART are read from `b`
ClickHouse	Database `b`: MergeTree, `ORDER BY` the primary key, e.g. `(l_orderkey, l_linenumber)`	Database `d`: LINEITEM `ORDER BY (l_shipdate, l_orderkey, l_linenumber)`; ORDERS and PART copied
How a query selects it	PostgreSQL `search_path = b`; ClickHouse database `b`	PostgreSQL `search_path = d,b`; ClickHouse database `d`

Because the table names are the same in both namespaces, the query files never change between designs. A BRIN index is used in PostgreSQL rather than a B-tree on ship date because, like ClickHouse's sparse index, it stores min/max values per block range and only works on physically ordered data.

Loading

`scripts/load_test.sh <SF>` does the whole load for one scale factor, one engine at a time:

1. Recreates both containers and drops any existing tables.
2. **PostgreSQL:** `schema.sql`; server-side `COPY … FROM '/data/sf<N>/<table>.tbl' WITH (DELIMITER '|')`; `post_load.sql` (primary keys after loading, then `VACUUM ANALYZE` for statistics and the visibility map); `design_D.sql`.
3. **ClickHouse:** `schema.sql`; `INSERT … FORMAT CSV` through `clickhouse-client` with `--format_csv_delimiter='|'` and `--input_format_csv_trim_whitespaces=0` (keeps trailing spaces in comments, so the stored text is byte-identical to PostgreSQL's); `OPTIMIZE TABLE … FINAL` (one part per table, so part count does not vary between sizes); `design_D.sql`.
4. Prints a fingerprint per table: row count, `sum(l_quantity)`, `sum(l_extendedprice)`.

The two engines must print identical fingerprints for `b.lineitem` and `d.lineitem`. At SF 1: 6,001,215 rows · 153,078,795 · 229,577,310,901.20.

Loading SF 1 took roughly 18 s (PostgreSQL `COPY`) and 9 s (ClickHouse `INSERT`), plus post-processing and design D. These are shell timings, not official measurements: the two engines receive the file by different paths.

Sizes and the design D check

`scripts/sizes.sh <SF>` writes `results/sizes_sf<N>.txt` with: the correlation between physical order and `l_shipdate` in PostgreSQL (about 1 for `d`, about 0 for `b`), heap and total size per PostgreSQL table, compressed and uncompressed size per ClickHouse table, and compressed size per LINEITEM column.

Query workload and validation

Four TPC-H queries with the specification's validation parameters, written once per engine; the two versions differ only in literal syntax and two explicit type casts, and they return identical results.

Query	What it does	LINEITEM columns read	Join	Parameters
Q1	Filter on ship date, group by return flag and line status, 8 aggregates	7	—	ship date ⩽ 1998-12-01 − 90 days

Query	What it does	LINEITEM columns read	Join	Parameters
Q6	Selective filter, one sum	4	—	1994; discount 0.05–0.07; quantity < 24
Q12	Filter on dates and ship mode, join, grouped counts	5	ORDERS	MAIL, SHIP; receipt date in 1994
Q14	Filter on one month, join, ratio of two sums	4	PART	ship date in September 1995

Differences between the two dialects

Construct	PostgreSQL	ClickHouse	Why
Date literal	`DATE '1994-01-01'`	`toDate('1994-01-01')`	Syntax
Date arithmetic (Q1)	`DATE '1998-12-01' - INTERVAL '90 days'`	`toDate('1998-12-01') - INTERVAL 90 DAY`	Syntax
Decimal literal (Q6)	`BETWEEN 0.05 AND 0.07`	`BETWEEN toDecimal32('0.05', 2) AND toDecimal32('0.07', 2)`	Makes the literal type explicit; same meaning
Ratio (Q14)	`100.00 * sum(…) / sum(…)`	`toDecimal128(100, 12) * sum(…) / sum(…)`	ClickHouse decimal division keeps the left operand's scale; without the cast the result is truncated to 4 decimals

Joins are written `lineitem JOIN orders` / `lineitem JOIN part` in both engines. PostgreSQL's planner chooses the join order itself; ClickHouse builds its hash table from the right-hand table, so the smaller table goes on the right.

Validation

`scripts/check_queries.sh` runs each query on both engines (design B) and prints the results next to the TPC-H reference answers in `sql/answers/`. The output for SF 1 is in `results/check_sf1.txt`.

Acceptance rule: sums and counts identical; averages and the Q14 ratio equal within a relative tolerance of 10^{-6}. Reference answers exist only for SF 1; at larger sizes the two engines are compared with each other.

Result at SF 1: all four queries match the reference and agree between engines.

Query	Reference	Both engines
Q1	4 groups, e.g. A/F sum_qty 37734107.00, count 1478493	identical sums and counts; averages agree to about 15 digits
Q6	123141078.23	123141078.2283
Q12	MAIL 6202 / 9324, SHIP 6200 / 9262	identical
Q14	16.38	16.3807786263955401

Q14 initially failed: ClickHouse returned 16.3807, truncated, about $5 \cdot 10^{-6}$ away from PostgreSQL. The explicit `toDecimal128` cast fixed it. It applies only to the two final aggregated values, so it does not affect execution time.

Measurement

`scripts/run.py` turns report Section 5.4 into one command per scale factor; every timed query execution becomes one row in `results/runs.csv`.

```
source .venv/bin/activate
python scripts/run.py --sf 1 --tag final --cold 5 --rounds 30
```

Options

Option	Default	Meaning
`--sf`	required	Scale factor currently loaded; checked against the LINEITEM row count
`--tag`	`pilot`	Campaign label written on every row, e.g. `pilot` or `final`
`--engines`	`pg ch`	Engines to measure, in order
`--designs`	`B D`	Physical designs to measure
`--queries`	`q1 q6 q12 q14`	Queries to run
`--cold`	5	Cold repetitions per query (design B only)

Option	Default	Meaning
`--warmup`	3	Discarded warm-up runs per query
`--rounds`	10	Warm round-robin rounds; we use 30 (see pilot findings)
`--mem-limit-gb`	8	Expected container memory limit; checked against the cgroup
`--compose-file`	`docker-compose.yml`	Repeatable; adds an override file, e.g. for the 4 GB memory-limit check

What happens for each engine

1. Stops the other engine's container, starts this one (`docker compose up -d`).
2. **Preflight:** checks that `b.lineitem` has about 6,001,215 × SF rows and that the container's `memory.max` equals `--mem-limit-gb`. Stops on a mismatch.
3. Dumps the engine version and settings to `env/`.
4. Starts a background sampler: every second it reads `cpu.stat` (`usage_usec`) and `memory.current` from the container's cgroup and appends to `results/resources.csv`.
5. **Cold runs, design B only.** For each repetition and each query, in rotated order: reset caches → open a new connection and run `SELECT 1` (not timed, so connection and catalog loading are excluded) → time the query.
6. **Warm block, for each design.** Reset caches once → 3 warm-up runs per query (recorded with `cache=warmup`, never analysed) → `--rounds` rounds of Q1, Q6, Q12, Q14 in round-robin order (`cache=warm`). Round-robin spreads slow drifts of the host over all queries equally.
7. **Instrumented run, untimed.** PostgreSQL: `EXPLAIN (ANALYZE, BUFFERS, SETTINGS)`; ClickHouse: `EXPLAIN indexes = 1`. Saved to `results/plans/<tag>_<engine>_<design>_sf<N>_<query>.txt`.
8. ClickHouse only: flushes its query log and copies, for every run's `query_id`, the rows and bytes read, duration and memory to `results/ch_query_log.csv`.
9. Stops the container.

One timed run

1. Read the container's `io.stat` and sum `rbytes` over devices.
2. Start the clock (`time.perf_counter`), send the query, fetch the full result, stop the clock.

3. Read `io.stat` again; the difference is the bytes this run read from storage.

Timing is client-side wall-clock time from submission until the whole result is received. All queries return at most four rows, so transfer time is negligible. The storage counter is kept by the Linux kernel for the container, in the same way for both engines; this is what classifies a run as in-memory (about 0 bytes read in warm runs) or not.

Connections: PostgreSQL via `psycopg` 3 with `autocommit` and `prepare_threshold=None` (no automatic prepared statements, so every run is planned like a plain query), schema chosen with `-c search_path`. ClickHouse via `clickhouse-connect` over HTTP (port 8123, no compression), database chosen per design, each run tagged with a unique `query_id`.

Cache reset

`scripts/reset_caches.sh pg|ch`:

1. ClickHouse only: `SYSTEM FLUSH LOGS`, so no query-log entry is lost.
2. `docker stop` the container, which empties the engine's own caches (PostgreSQL shared buffers; ClickHouse mark cache, query condition cache and others).
3. `sync`, then `echo 3 | sudo -n tee /proc/sys/vm/drop_caches`, which empties the Linux page cache.
4. `docker start` the container. `run.py` then retries a connection every second until the engine answers.

While ClickHouse restarts, its driver logs `Unexpected Http Driver Exception` for each failed attempt. This is harmless; a line in `run.py` (`logging.getLogger("clickhouse_connect").setLevel(logging.CRITICAL)`) silences it.

The pilot confirmed the reset works under WSL2: cold runs read from storage exactly the data sizes measured by `sizes.sh`, and warm runs read 0 bytes.

Columns of `results/runs.csv`

Column	Content
`timestamp`	ISO time the row was written
`tag`	Campaign label
`engine`	`pg` or `ch`
`design`	`B` or `D`

Column	Content
`sf`	Scale factor
`mem_limit_gb`	Container memory limit
`query`	`q1`, `q6`, `q12` or `q14`
`cache`	`cold`, `warmup` or `warm`
`run_no`	Repetition or round number
`seconds`	Wall-clock time of the run
`io_read_bytes`	Bytes the container read from storage during the run
`rows_returned`	Rows in the result
`query_id`	Unique run identifier; joins with `ch_query_log.csv`

Rows are appended immediately, so an interrupted run keeps what it has measured. A full SF 1 run (5 cold, 30 warm rounds, both designs) takes about 15–20 minutes.

Analysis

`scripts/analyze.py` reads `results/runs.csv` for one campaign tag and applies the report's metrics and decision rules (Section 5.5, Table 6) automatically, so no outcome can be adjusted after seeing the data.

```
python scripts/analyze.py --tag final > results/analysis_final.txt 2>&1
```

Options: `--tag` (required), `--boot` (bootstrap resamples, default 10,000), `--seed` (default 42, so results are reproducible). Warm-up runs are always excluded.

What it computes

1. **Per group** (engine × design × SF × query × cache state): number of runs, median time, first and third quartiles, relative IQR = (Q3 − Q1) ÷ median, median bytes read from storage.
2. **Speedup** for each design × SF × query × cache state: S = median PostgreSQL time ÷ median ClickHouse time. S > 1 means ClickHouse is faster.
3. **95% confidence interval for S** by bootstrap: each engine's runs are resampled with replacement, the two medians and their ratio are recomputed 10,000 times, and the 2.5th and 97.5th percentiles form the interval.

4. **Change of S between consecutive sizes** present in the data: r = S(larger) ÷ S(smaller), with a bootstrap interval, and r per doubling of the data (r^(1/log$_2$ size ratio)), because the SF 4 → 10 step is 2.5×, not 2×.
5. **Decision table** (design B, warm runs, per query), following report Table 6:

Hypothesis	Supported	Not supported	Otherwise
H1	CI of S above 1 at all sizes	CI of S below 1 at any size	Inconclusive
H2a	CIs of r(1→2) and r(2→4) within [0.9, 1.1]	Either CI entirely outside the band	Inconclusive
H2b	CI of r(4→10) above 1.1, and the memory regimes as expected	CI of r(4→10) below 1.1 with the same regimes	Inconclusive

With fewer than four sizes, H1 reads "supported so far (partial)" and H2a or H2b "not testable yet". The H2b verdict carries a reminder to check the memory regimes from `io_read_bytes`: in warm runs, PostgreSQL should read from storage at SF 10 and ClickHouse should not.

Output files

File	Content
`results/summary_<tag>.csv`	Medians, quartiles, relative IQR and median storage reads per group
`results/speedups_<tag>.csv`	S with its confidence interval per design, SF, query and cache state
`results/ratios_<tag>.csv`	r between consecutive sizes (only when two or more sizes exist)
`results/decisions_<tag>.csv`	The decision table
`results/speedup_<tag>.png`	One panel per query: S against SF on a log axis, B warm, B cold and D warm, with confidence-interval bars

Reproducing everything

A teammate with a Windows machine (16 GB RAM, about 100 GB free on an SSD) can reproduce the SF 1 results in about two hours, most of it one-time setup.

One-time setup

1. **WSL2.** Create `C:\Users\<you>\.wslconfig` and `/etc/wsl.conf` as in Host setup, then in PowerShell run `wsl --update` and `wsl --shutdown`. If C: is short on space, move Ubuntu: `wsl --manage Ubuntu --move E:\WSL\Ubuntu`.
2. **Docker Engine inside Ubuntu.** Turn off Docker Desktop's WSL integration (including "Enable integration with my default WSL distro") and quit it, then:

   ```
   curl -fsSL https://get.docker.com | sudo sh
   sudo usermod -aG docker $USER
   sudo systemctl enable --now docker
   ```

   Open a new terminal; `docker info` must show Cgroup Driver systemd, Cgroup Version 2, Operating System Ubuntu.
3. **The repository.** Clone it into the Linux filesystem, never under `/mnt/c` or `/mnt/e`:

   ```
   cd ~ && git clone <repository URL> DB-Benchmark && cd DB-Benchmark
   ```

4. **Python environment.**

   ```
   sudo apt install -y python3-venv python3-pip build-essential git
   python3 -m venv .venv && source .venv/bin/activate
   pip install -r requirements.txt
   ```

5. **Password-free cache clearing:** the sudoers rule from Host setup.
6. **DBGen:** `cd ~ && git clone https://github.com/gregrahn/tpch-kit.git && cd tpch-kit/dbgen && make MACHINE=LINUX DATABASE=POSTGRESQL`.
7. **Check the containers:** `docker compose up -d postgres` and `docker compose up -d clickhouse` (one at a time), then compare their settings with the tables in Containers and engine configuration.

One experiment cycle (per scale factor)

Every terminal starts with `cd ~/DB-Benchmark && source .venv/bin/activate`. Replace `1` by the scale factor.

1. **Generate:**

   ```
   cd ~/tpch-kit/dbgen && ./dbgen -s 1 -f
   mkdir -p ~/DB-Benchmark/data/sf1 && mv lineitem.tbl orders.tbl part.tbl
   ~/DB-Benchmark/data/sf1/ && rm -f *.tbl
   cd ~/DB-Benchmark && sed -i 's/|$//' data/sf1/*.tbl
   ```

2. **Load and fingerprint:** `scripts/load_test.sh 1 > results/load_test_sf1.txt 2>&1`. Both engines must show identical counts and sums.
3. **Validate the queries:** `scripts/check_queries.sh > results/check_sf1.txt 2>&1`. All four queries must agree between engines (and with `sql/answers/` at SF 1).
4. **Record sizes:** `scripts/sizes.sh 1`.
5. **Pilot:** `python scripts/run.py --sf 1 --tag pilot --cold 3 --rounds 3`, then `python scripts/analyze.py --tag pilot`. Check that nothing failed and that warm `io_read_bytes` match the expected memory regime.
6. **Final measurement:** `python scripts/run.py --sf 1 --tag final --cold 5 --rounds 30` (keep the laptop plugged in and otherwise idle).
7. **Analyse:** `python scripts/analyze.py --tag final > results/analysis_final.txt 2>&1`. Rerun it after each new scale factor: it analyses every size present for that tag.
8. **Commit:** `results/` and `env/`, with a message naming the scale factor.
9. **Free disk space** before the next size: `docker compose down -v` deletes both database volumes; `rm -rf data/sf1` deletes the generated files.

Only one scale factor is loaded at a time, so the preflight check in `run.py` always matches the data on disk.

Status, decisions and next steps

SF 1 is fully done: validated, sized and measured; at that size ClickHouse is 3.8–6.7 times faster in warm runs. The detailed results and their implications for the report are in `SF 1 Pilot Findings and Report Implications`.

Key design decisions

Decision	Reason
WSL2 instead of native Linux	No native Linux machine available; WSL2 with cgroup v2 keeps the memory limit and disk counters the protocol needs
One engine running at a time	Neither engine competes for CPU, memory or page cache with the other
Same `cpuset` and 8 GB limit, no swap	Same compute and memory budget; the limit includes the page cache

Decision	Reason
Designs B and D	B isolates storage layout and execution; D measures how much data skipping contributes
BRIN, not B-tree, in PostgreSQL's design D	Closest equivalent to ClickHouse's sparse min/max index
Plain `String`, no `LowCardinality`	Avoid tuning one engine only
ClickHouse query and filter caches off	Repeated warm runs would otherwise skip work that PostgreSQL must redo
`SELECT 1` before each cold run	Excludes connection and catalog loading from cold timings
30 warm rounds instead of 10	Pilot variability above the report's 10% threshold
Raw results committed to Git	The report promises to publish them; every number is traceable

Troubleshooting

Symptom	Cause	Fix
`stat -fc %T /sys/fs/cgroup` prints `tmpfs`	cgroup v1	Add `kernelCommandLine=cgroup_no_v1=all` to `.wslconfig`, `wsl --shutdown`
`which docker` shows `/mnt/c/Program Files/...`	Windows Docker Desktop binary found	`appendWindowsPath=false` in `/etc/wsl.conf`; install Docker Engine in Ubuntu
`docker compose` uses files from `/mnt/e/...`	Terminal open in an old copy of the project	`cd ~/DB-Benchmark`; check the prompt
`extra data after last expected column` in `COPY`	DBGen ends every line of the `.tbl` files with a pipe delimiter	`sed -i 's/|$//' data/sf<N>/*.tbl`
`could not resize shared memory segment`	`/dev/shm` too small	`shm_size: 2560m` in the compose file
`Unexpected Http Driver Exception` during runs	ClickHouse restarting after a cache reset	Harmless; silenced by the logging line in `run.py`

Symptom	Cause	Fix
`Cache reset failed`	Sudoers rule missing	Install the rule from Host setup; test with `sudo -k` then `sudo -n`
`pip install -r requirements.txt` prints nothing	Empty downloaded file	Recreate `requirements.txt`; strip Windows line endings with `sed -i 's/\r$//'`
`preflight` stops with a row-count error	A different scale factor is loaded	Load the right SF with `load_test.sh`
ClickHouse result differs in the 5th decimal	Decimal division keeps the left operand's scale	Cast the left operand, as in Q14

Next steps

1. Pilot at SF 10 with 3 runs: check that loading and queries fit in 8 GB, and that in warm runs PostgreSQL reads from storage while ClickHouse does not (the precondition for testing H2b).
2. Generate, load, validate and measure SF 2, 4 and 10 with the cycle above, then run `analyze.py --tag final` over all sizes.
3. Memory-limit check at SF 4: same data with a 4 GB limit, through a `docker-compose.mem4g.yml` override and `--mem-limit-gb 4`.
4. Small diagnostic runs: compare PostgreSQL plans for Q1 and Q12 under B and D; Q1 and Q6 with floating-point types; `SELECT 1` latency per engine.
5. Apply the report revisions and build the M2 presentation from `results/`.